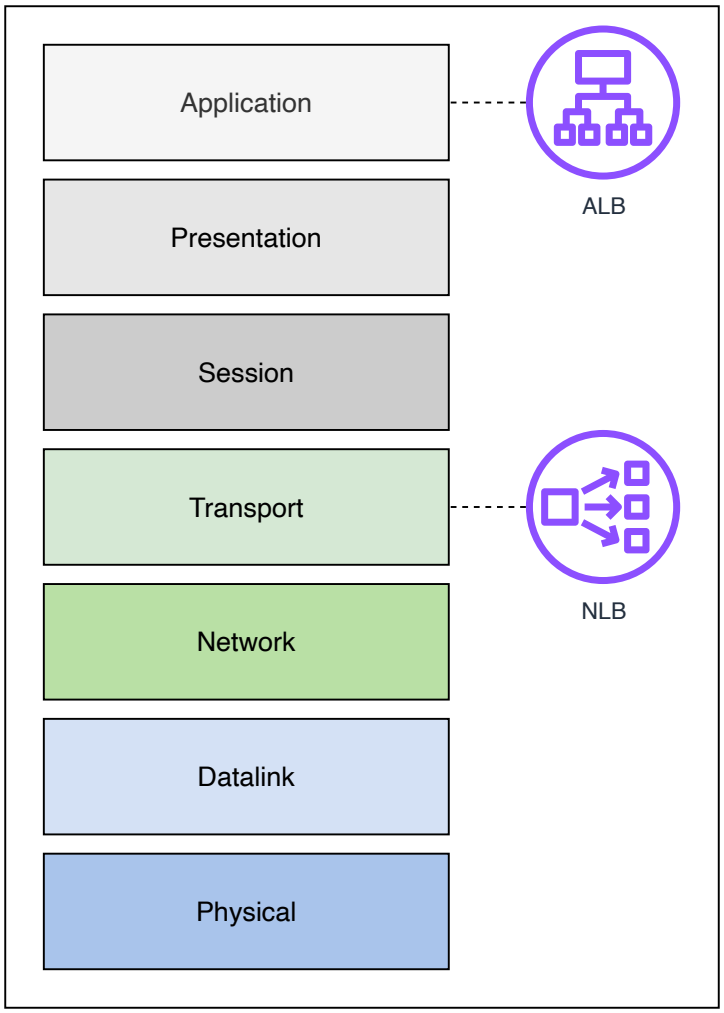
Network Load Balancer (NLB)

Understand the role of AWS Network Load Balancers in designing high-throughput, low-latency applications. Learn deployment, configuration, monitoring, and troubleshooting techniques essential for scalable cloud-native solutions, preparing you for the AWS Developer Associate exam.

The **Network Load Balancer (NLB)** operates at **layer 4 (transport layer)** of the OSI model, which enables it to handle millions of requests per second with incredibly low latencies. Unlike the application load balancer optimized for HTTP/HTTPS, NLB shines in scenarios that demand raw network performance.

Ask



NLB and ALB in the OSI Model

Understanding the network load balancer (NLB) is essential for multiple exam domains of the AWS Certified Developer – Associate (DVA-C02) exam, particularly deployment and troubleshooting. This lesson focuses on NLB's role in designing scalable, high-performance, cloud-native applications. You'll encounter questions that assess your ability to configure and monitor NLBs, evaluate when to use them over application load balancers (ALBs), and ensure optimal performance using metrics and health checks.

Understanding high-throughput use cases

NLB is specifically designed for workloads demanding persistent, low-latency connections and the ability to manage an immense number of concurrent

Ask

sessions. It operates at layer 4 (the transport layer) of the OSI model, which allows it to handle traffic at a very high throughput with minimal latency.

Consider applications where real-time performance is critical:

- **Multiplayer gaming platforms:** These require extremely low latency to ensure smooth gameplay and synchronized experiences across all players. NLB's direct pass-through of TCP connections minimizes processing overhead, delivering the speed needed.
- **High-frequency stock trading applications:** Even microseconds of delay can result in significant financial losses. NLB's consistent performance and ability to handle millions of requests per second make it ideal for such sensitive financial systems.

For instance, a large-scale IoT application with thousands of devices continuously transmitting sensor data over TCP needs both consistent performance and massive scalability. The NLB provides this by efficiently handling a vast number of persistent connections without inspecting application-layer content.

Similarly, services that rely on raw TCP or UDP protocols, such as:

- SMTP (Simple Mail Transfer Protocol) for email transmission
- FTP (File Transfer Protocol) for file transfers
- DNS (Domain name system) for name resolution (which often uses UDP)

These protocols benefit from NLB's native support and its capability to maintain high throughput and low latency without interfering with the application layer. In all these cases, NLB ensures scalable and stable communication by simply forwarding traffic to healthy targets, making it the preferred choice for such demanding scenarios.

Ask

Deploying and configuring NLB

To deploy an NLB in a production-grade environment, you'll often automate using tools like AWS CLI or CloudFormation. As a developer preparing for the DVA-C02 exam, you should understand how these steps translate into real-world CI/CD and infrastructure workflows.

- You begin by choosing the load balancer type as "network." Then, select your scheme (internet-facing or internal), assign subnets in multiple availability zones, and configure listeners. These listeners define the port and protocol used for incoming traffic—commonly TCP 80 or 443.
- Next, you associate a target group with your NLB. This group holds the resources that will serve the requests—typically EC2 instances, Lambda functions, or IP addresses. NLB can distribute traffic evenly across targets, monitor health, and react to target state changes.

Let's look at an CLI example of creating a target group and registering it:

main.sh Shell-20

```
aws elbv2 create-target-group \
    --name my-targets \
    --protocol TCP \
    --port 80 \
    --vpc-id vpc-0123456789abcdef0

aws elbv2 register-targets \
    --target-group-arn arn:aws:elasticloadbalancing:region:account-id
    --targets Id=i-0123456789abcdef0
```

Creating target groups and register targets using the AWS CLI

Ask

Working with static and elastic IPs

In regulated environments or integrations with external systems, you may need a predictable IP address to whitelist in firewalls or partner systems. NLB supports static IP addresses by default—assigning one per subnet—and also allows associating Elastic IPs for even more control. This makes it suitable for use cases requiring IP stability, unlike ALB, which uses dynamic IPs.

Scaling with Auto Scaling groups

When combined with Auto Scaling groups (ASGs), NLB becomes a dynamic component of an elastic architecture. As your application's demand increases or decreases, ASG adjusts the number of backend instances, and the NLB updates the target group membership automatically.

Health checks ensure only healthy instances receive traffic. You can fine-tune these checks to evaluate not just the server's availability but also application readiness. Additionally, configuring a deregistration delay helps gracefully drain connections before removing an instance.

Observing performance with CloudWatch

As a developer, being able to interpret CloudWatch metrics is critical for troubleshooting and optimizing application behavior. Metrics like `ActiveFlowCount` and `ProcessedBytes` help us decide when to scale, investigate connection surges, or fine-tune backend performance.

Monitoring is crucial to ensure your NLB is functioning correctly under load. Amazon CloudWatch offers a suite of metrics for this purpose:

- `ActiveFlowCount` reflects the number of concurrent TCP flows. A rising trend may indicate growing demand or slow session teardown.
- `NewFlowCount` tells you how many connections are being initiated ov which helps gauge spikes or unusual traffic behavior.

Ask

- `ProcessedBytes` tracks data throughput and helps you visualize bandwidth usage.

You can configure alarms based on these metrics to trigger automated responses or alerts when thresholds are breached.

Comparing NLB with ALB

Though both are part of AWS Elastic Load Balancing, NLB and ALB differ substantially in purpose and capability. NLB functions at layer 4, directing traffic based on IP protocol data and offering static IP addresses, ideal for backend services or systems with IP-based whitelisting. ALB, on the other hand, works at layer 7 and understands HTTP/HTTPS requests, allowing it to make intelligent routing decisions based on paths, headers, and hostnames.

If your application requires content-based routing or WebSocket connections, ALB is the better fit. But for raw speed, simple TCP handling, and stable IP behavior, NLB is unmatched.

Troubleshooting and diagnostics

Developers are expected to identify and resolve traffic or health issues without relying entirely on network teams. For the exam and real-world application, pay attention to tools like VPC flow logs, CloudWatch logs, and ELB access logs. These offer visibility into every step of the request life cycle—from client connection to target response.

Effective troubleshooting begins with verifying that the NLB's targets are marked healthy. If not, double-check security group rules, listener port configurations, and that the backend app is properly responding.

Next, examine the network layer: ensure subnets and route tables allow tra flow correctly. Use CloudWatch metrics like `TargetResponseTime` and `HealthyHostCount` to identify potential lags or failures. For deeper investigation,

Ask

enable VPC flow logs to trace traffic, or use ELB access logs to study request-level details.

Conclusion

This lesson provided a complete walk-through of the AWS Network Load Balancer (NLB), starting with its purpose and role in high-performance, low-latency applications that rely on TCP and UDP traffic. You explored NLB's advantages over Application Load Balancers (ALBs), learned to deploy and

Ask